

Selection with if and else

CSC103 Programming Fundamentals / Lecture 08

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Classify an integer as negative, zero or positive. Draw a decision outline, then write an if-else chain.

Check the negative case first.

Learning objectives

- Implement one-way and two-way selection
- Build an ordered multi-branch decision
- Test boundaries and nested conditions

CLO-2

The if statement

Two-way selection

Multiple selections

Nesting and compound statements

The if statement

- An if condition must be Boolean.
- The body executes only when the condition is true.
- Braces clearly group all statements in the branch.

Example

```
if (balance < 0) {  
    System.out.println("Overdrawn");  
}
```

Two-way selection

- An if-else chooses exactly one branch.
- Statements after the structure execute after either branch completes normally.

Example

```
if (marks >= 50) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

Multiple selections

- An else-if chain selects the first matching branch.
- Order overlapping thresholds from most restrictive to least restrictive.
- Independent if statements can execute several bodies.

Example

```
if (score >= 80) {  
    band = "High";  
} else if (score >= 50) {  
    band = "Middle";  
} else { band = "Low"; }
```

Nesting and compound statements

- A nested if expresses a decision inside another decision.
- An else attaches to the nearest unmatched if.
- Braces prevent misleading indentation.

Example

```
if (registered) {  
    if (paid) {  
        System.out.println("Admit");  
    }  
}
```

Branch order determines the selected result

- An else-if chain stops testing after its first matching condition.
- Overlapping ranges need an order that gives the most specific case priority.
- A final else covers every case not selected earlier.

Example

If `>=50` comes before `>=80`, a score of 90 selects the `>=50` branch immediately.

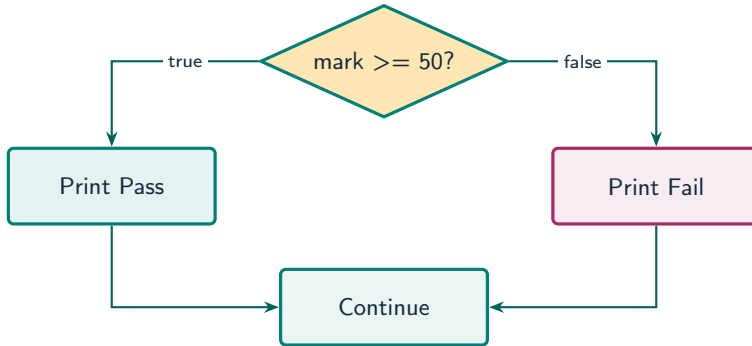
Independent tests and exclusive alternatives

- Separate if statements can execute more than one body.
- An if-else chain expresses mutually exclusive outcomes.
- For sign classification, exactly one of negative, zero and positive is appropriate.

Example

```
if (n < 0) ...  
else if (n == 0) ...  
else ...
```

A decision selects one path



What to notice

Exactly one branch executes; both normal paths rejoin after the selection.

Key distinctions

Input n	$n < 0$	$n == 0$ if reached	Chosen result
-1	true	Not evaluated	Negative
0	false	true	Zero
1	false	false	Positive

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int score = 80;
String band;
if (score >= 80) { band = "High"; }
else if (score >= 50) { band = "Middle"; }
else { band = "Low"; }
System.out.println(band);
```

First example: result

High

The first condition is true.

The remaining branches are skipped.

Multiple selections

Nesting and compound statements

Guided practice

Classify an integer as negative, zero or positive. Draw a decision outline, then write an if-else chain.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Check the negative case first.
- If the number is not negative, test equality to zero.
- Every remaining integer is positive.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution08 {  
    public static void main(String[] args) throws Exception {  
        int n = 0;  
        if (n < 0) {  
            System.out.println("Negative");  
        } else if (n == 0) {  
            System.out.println("Zero");  
        } else {  
            System.out.println("Positive");  
        }  
    }  
}
```

Execution trace

Execution point	Condition or action	Result
First test	$0 < 0$	false
Second test	$0 == 0$	true
Selected body	Print Zero	Zero
Remaining branch	Skip else	No extra output

Follow the changing state in execution order.

Solution result

Zero

Why this result follows
Every remaining integer is positive.

A common incorrect approach

```
if (marks >= 50);  
{ System.out.println("Pass"); }
```

Example

The semicolon creates an empty `if` body. Remove it so that the block belongs to the `if`.

Boundary and failure checks

Check the stated input assumptions.

Write a shipping-charge decision for three weight ranges.

State your rules and test each boundary.

```
if (n < 0) ... else if (n == 0) ... else  
...
```

Test -1, 0 and 1. Exactly one label must appear.

Check your understanding

How many branches can an else-if chain select? Why test 49, 50 and 51?

Write a prediction and explain the rule that supports it.

Answer and explanation

At most one. Those cases exercise both sides of the 50 boundary and equality itself.

The semicolon creates an empty if body. Remove it so that the block belongs to the if.

Compare cases and predict outcomes

Mark	Condition	Printed result
49	false	Fail
50	true	Pass
51	true	Pass

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a grade band selection: 80 or above is High, 50 through 79 is Pass, and below 50 is Fail. Explain the order of tests.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
if (mark >= 80) {  
    System.out.println("High");  
} else if (mark >= 50) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

- The narrower, higher threshold must be tested first.
- After `mark >= 80` is false, `mark >= 50` means 50 through 79.
- The example assumes a valid mark; range validation is a separate step.

Overtime payroll

- The first 40 hours use the regular rate. Only excess hours use the 1.5 multiplier.
- At exactly 40 hours, overtime hours are zero.
- This is the fictional business rule supplied by the exercise, not an employment-law rule.

Source: supplied ISB campus slides. See speaker notes and [ISB_Source_Map.md](#).

Worked problem: Overtime payroll

Task

Compute pay for 45 hours at 20 units per hour using the source overtime rule.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture08 {  
    public static void main(String[] args) throws Exception {  
        double hours = 45, rate = 20;  
        double pay;  
        if (hours > 40) {  
            pay = 40 * rate + (hours - 40) * rate * 1.5;  
        } else {  
            pay = hours * rate;  
        }  
        System.out.printf(java.util.Locale.US, "Pay: %.2f%n", pay);  
    }  
}
```

Worked trace and expected result

Hours	Regular + overtime pay	Total
39	780 + 0	780
40	800 + 0	800
45	800 + 150	950

Expected console output

Pay: 950.00

Extend the ISB example

Practice

What is wrong with multiplying all 45 hours by 1.5 times the rate?

Explain your reasoning

It applies the overtime premium to regular hours. That incorrect formula gives 1350 instead of 950.

Lecture summary

- one-way and two-way selection
 - an ordered multi-branch decision
 - boundaries and nested conditions
- Check the negative case first.
 - If the number is not negative, test equality to zero.
 - Every remaining integer is positive.

After-class practice

Write a shipping-charge decision for three weight ranges.
State your rules and test each boundary.

Syllabus reading
Deitel Ch 7 (as listed)
Next lecture: Short-circuiting and switch